HACKATHON TEAM FORMATION & JUDGING SYSTEM

A PROJECT REPORT

Submitted by

VISHAL S

(240701600)

in partial fulfilment of the End Semester Examination for the course

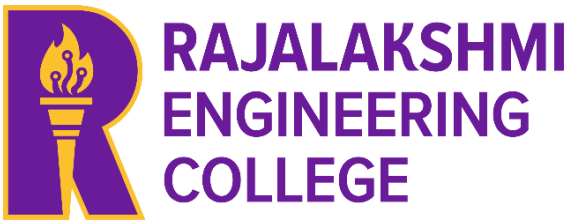
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "Hackathon Team Formation & Judging System" is the bonafide work of Vishal S (240701600), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. Jinusophia
COURSE FACULTY
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Vishal S (240701600), hereby declare that the project report titled “Hackathon Team Formation & Judging System”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date: _____

SIGNATURE OF THE STUDENT

Vishal S

240701600

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. M. Abhay Shankar for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinusophia, Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and the Zoho Creator learning resources for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

VISHAL S

ABSTRACT

Technical hackathons organised in engineering colleges are still administered largely through form links, messaging groups and spreadsheets. Participant details, team formation, judge assignments and score sheets are kept in separate files, which leads to duplicate teams, scores entered outside the allowed range, the same judge scoring a team twice, arithmetic errors in totals, and a leaderboard that is prepared by hand after the event. Organisers have no single place in which to see how many teams are registered, which teams are approved or how the judging is progressing.

This project presents the Hackathon Team Formation & Judging System, a low-code application developed on Zoho Creator. The data model consists of four forms (Participant, Judge, Team and Scoring) connected by lookup fields, with a formula field that totals the four judging criteria automatically. Five list reports and one dashboard page with two charts give organisers, judges and participants a live view of the event. Seven Deluge scripts and an approval process automate the rules of the event: scores outside 0 to 10 and duplicate teams or duplicate judge entries are blocked at validation time, and e-mail alerts are sent to the application administrator when a participant registers, a team is approved or a score is submitted. A weekly scheduled reminder prompts judges to complete scoring. A Keyword Extraction AI field tags every team's problem statement with its key terms. Access is organised through the Judge and Participant roles and two permission sets.

The application was tested with twelve participants, ten judges, twelve teams and fourteen score records. Four negative tests (score above 10, duplicate score, duplicate team and missing e-mail) were all rejected correctly, the formula totals matched manual calculation, and the leaderboard and dashboard charts agreed with the underlying records. The low-code approach removed manual totalling, removed the need for a separate spreadsheet and kept the whole application maintainable through configuration and short scripts. The report also records the limits met on the platform, including the absence of separate Stage and Production environments in the trial account used.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Hackathon Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	Hackathon Team Formation & Judging System
Application Domain	Education – college hackathon management
Student Name & Register No.	Vishal S – 240701600
Zoho Creator Edition	Zoho Creator (free trial edition, India data centre)
Application Name (Link Name)	hackathon-team-formation-judging-system
Published / Live URL	https://creatorapp.zoho.in/vishalabs12/hackathon-team-formation-judging-system/
Demo Credentials (per role)	Administrator: application owner account (vishalabs12@gmail.com) | Judge and Participant roles are defined; no additional user accounts were invited
Number of Forms / Subforms	4 forms / 0 subforms
Number of Reports (by type)	List: 5 | Kanban: 0 | Calendar: 0 | Pivot/Chart: 2 (dashboard charts) | Others: 0
Number of Pages / Dashboards	1 (Hackathon Dashboard)
Number of Workflows	Form: 6 | Schedule: 1 | Approval: 1 | Blueprint: 0 | Batch: 0
Deluge Functions / Custom Actions	7 Deluge scripts (6 form-workflow scripts and 1 schedule script); 0 stored functions
Integrations	Zoho Mail (sendmail task), Zoho Zia Keyword Extraction AI field
Roles & Profiles Defined	Administrator (CEO), Judge, Participant; permission sets: Judge Access, Participant Access
Environments Used	Single application workspace; separate Stage and Production environments were not configured
Date of Production Deployment	Not separately deployed; live application URL used for demonstration

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7
REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 10
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 11
SYSTEM DESIGN 13
4.1 System Architecture 13
4.2 Data Model 13
4.3 Form Design 15
4.4 Process and Workflow Design 17
4.5 User Interface and Navigation Design 18
4.6 Access Control Design 18
IMPLEMENTATION 19
5.1 Application Setup 19
5.2 Forms and Data Capture 19

5.3 Reports 20
5.4 Pages and Dashboards 21
5.5 Deluge Scripting 22
5.6 Workflows and Automation 24
5.7 Integrations 25
5.8 User Management and Role-Based Access 26
5.9 AI and Analytics Features 27
5.10 Pro-Code Extension 28
5.11 Mobile and Portal Access 28
TESTING AND DEBUGGING 29
6.1 Testing Strategy 29
6.2 Test Cases 29
6.3 Debugging Deluge Scripts 31
6.4 Defect Log 31
DEPLOYMENT, SECURITY AND GOVERNANCE 33
7.1 Environments and Deployment Strategy 33
7.2 Performance Optimisation 34
7.3 Security Features 35
7.4 Governance and Administration 36
RESULTS AND DISCUSSION 37
8.1 Application Output 37
8.2 Achievement of Objectives 39
8.3 Comparison with the Existing System 39
8.4 Limitations of the Low-Code Approach 40
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 40
CONCLUSION AND FUTURE ENHANCEMENTS 42
9.1 Conclusion 42
9.2 Future Enhancements 42
9.3 Learning Outcomes and Career Relevance 43
REFERENCES 44
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 45
APPENDIX B – COURSE OUTCOME MAPPING 48
APPENDIX C – COMPLETE DELUGE SOURCE CODE 49
APPENDIX D – USER MANUAL 51
D.1 Accessing the Application 51
D.2 Instructions for the Administrator 51
D.3 Instructions for the Judge 51

D.4 Instructions for the Participant 52
APPENDIX E – EVALUATION RUBRIC 53
APPENDIX F – VIVA-VOCE RECORD SHEET 55

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 5
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 10
Table 3.4: Hardware and Software Requirements 10
Table 3.5: Use Case Description – Register Participant 11
Table 3.6: Use Case Description – Submit Team Score 12
Table 3.7: Use Case Description – Approve Team 12
Table 4.1: Entities and Relationships 14
Table 4.2: Field Specification – Participant Form 15
Table 4.3: Field Specification – Judge Form 15
Table 4.4: Field Specification – Team Form 16
Table 4.5: Field Specification – Scoring Form 16
Table 4.6: Role–Permission Matrix 18
Table 5.1: Reports Implemented 20
Table 5.2: Deluge Scripts Implemented 22
Table 5.3: Workflows Implemented 24
Table 5.4: Integrations Implemented 25
Table 6.1: Test Cases 29
Table 6.2: Defect Log 31
Table 7.1: Deployment Checklist 33
Table 7.2: Performance Optimisation Results 35
Table 7.3: Security Controls Implemented 35
Table 8.1: Mapping of Objectives to Outcomes 39
Table 8.2: Existing System versus Proposed Application 39
Table A.1: Feature Coverage Checklist 45
Table B.1: Mapping of Course Outcomes to Report Chapters 48
Table E.1: Evaluation Rubric for the Individual Project 53
Table E.2: Performance Level Descriptors 54
Table F.1: Viva-Voce Questions and Responses 55
Table F.2: Marks Summary 55

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 4
Figure 3.1: Use Case Diagram of the Application 11
Figure 4.1: System Architecture of the Application 13
Figure 4.2: Entity–Relationship Diagram 14
Figure 4.3: Workflow / Process Flow for Score Submission and Validation 17
Figure 4.4: Navigation Structure and Wireframe of the Hackathon Dashboard Page. 18
Figure 5.1: Application Home and Navigation Sections (All Participants Report) 19
Figure 5.2: Team Form with the Problem Keywords AI Field in the Form Builder 20
Figure 5.3: Scoring Form in Live Mode 20
Figure 5.4: Leaderboard Report 21
Figure 5.5: Hackathon Dashboard Page 22
Figure 5.6: Score Submitted Alert Workflow Configuration 25
Figure 5.7: E-mail Integration Flow from Trigger to Administrator 26
Figure 5.8: Roles and Permission Sets Configuration 27
Figure 5.9: AI Keyword Extraction in the All Teams Report 28
Figure 6.1: Deluge Error for the Composite Name Field 31
Figure 7.1: Build, Test, Release and Maintain Cycle Used in Place of Separate Environments 33
Figure 7.2: Governance Controls Applied to the Application 36
Figure 8.1: Scenario Step 1 – Participants Registered in the All Participants Report. 37
Figure 8.2: Scenario Step 2 – Judges Recorded in the All Judges Report 38
Figure 8.3: Scenario Step 3 – Dashboard Charts After Scoring 38

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DS	Deluge Script (script identifier used in this report)
ER	Entity–Relationship
FR	Functional Requirement
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
TC	Test Case
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

A hackathon is a time-bound event in which students form teams, build a working solution to a stated problem and present it to a panel of judges. Colleges and student chapters conduct such events every semester, and a typical event involves a few dozen participants, several faculty or industry judges and an organising committee. The organiser has to register participants, form and approve teams, record each team's problem statement, assign judges, collect scores against agreed criteria and announce a ranked result.

In most colleges this work is done with a registration form, a spreadsheet and printed score sheets. The people involved are the organising committee (who register and approve teams), the judges (who evaluate each team) and the participants (who need to know the status of their team). This project builds an application for that process on the Zoho Creator low-code platform, so that registration, team approval, scoring and result display are handled in one place with rules enforced by the system rather than by manual checking.

1.2 Problem Statement

The manual handling of hackathon data is slow and error-prone because registration, team details and scores live in separate files that are joined by hand. There is no automatic check for duplicate teams, invalid scores or repeated judging, and no live view of the results. The problem addressed in this project is to provide a single, rule-enforcing application that captures participants, teams, judges and scores, automates the routine checks and notifications, and presents the leaderboard instantly.

- Team and score data are duplicated across forms, spreadsheets and messages, so the same team can be recorded twice under the same name.
- Scores are written and totalled by hand; a score outside the agreed 0 to 10 range or a wrong total is noticed only after the event.
- A judge can accidentally score the same team twice, and there is no way to tell which judges have not yet scored.

- Approval of team registrations, notification of the organiser and the preparation of the leaderboard are manual steps with no audit trail.
- Problem statements are read one by one to decide the domain of a team, which makes matching teams to judge expertise slow.

1.3 Objectives

- To design a normalised data model and forms for participants, judges, teams and scores, connected by lookup fields.
- To automate score validation, duplicate prevention, e-mail alerts and judge reminders using Deluge scripting and workflows.
- To provide role-based access, dashboards and reports for the organiser, judges and participants.
- To integrate the application with Zoho Mail for notifications and with Zoho Zia (Keyword Extraction) for tagging problem statements.
- To test the application with realistic data and document its security, deployment and governance controls.

1.4 Scope of the Project

In scope: registration of participants and judges; creation and approval of teams; scoring of each team by judges on four criteria (Innovation, Technical, Presentation and Impact, each out of 10); automatic totals; five list reports; a dashboard with a leaderboard and two charts; e-mail alerts to the administrator; a weekly judge reminder; the Judge and Participant roles with two permission sets; and an AI Keyword Extraction field on team problem statements. The platform is the Zoho Creator web application.
Out of scope: online payment, certificate generation, project file submission, integration with external APIs or Zoho CRM, a customer portal, the Zoho Creator mobile application, and e-mail delivery to participants or judges (all e-mail alerts go only to the application administrator).

1.5 Justification for a Low-Code Approach

Low-code platforms let developers build applications with visual modelling and a small amount of scripting instead of hand-written code for every layer [1]. The term was introduced by Forrester Research in 2014 [2], and analysts now treat low-code application platforms as a mainstream category [3]. Zoho Creator provides forms,

reports, pages, workflows, approval processes, a scripting language (Deluge), hosting and role-based access in one environment [4]. For this project that means the data model, validation rules, e-mail alerts and dashboard could be configured and tested directly in the browser, with no separate database, server or front-end framework to set up. The application can be changed later by the organiser team by editing a form or a short script, which suits an event whose rules change every semester. The trial edition used here costs nothing, which is also appropriate for a student project.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing process, followed by most college hackathons, is described below. Participants register through a Google Form or a messaging group. The organiser copies the entries into a spreadsheet and forms the teams, checking for duplicate team names by eye. Problem statements are read one by one so that teams can be tagged by domain. Judges receive printed score sheets and write marks by hand. The organiser then re-types the marks, totals them on a calculator or spreadsheet, prepares the leaderboard and shares the result by e-mail or messaging group.

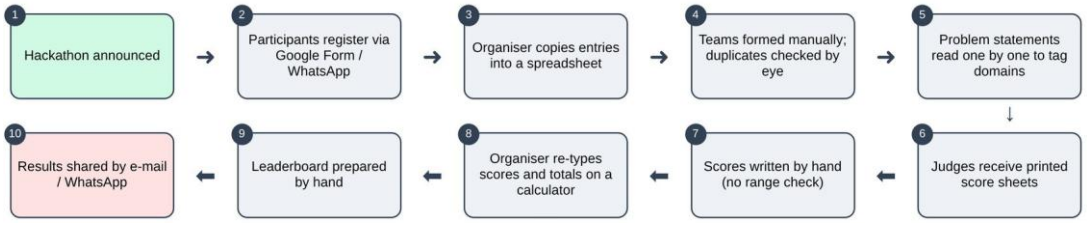


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Data is entered more than once (form, spreadsheet, score sheet and result sheet), which causes transcription errors.
- There is no validation: out-of-range scores, duplicate teams and repeated judging are not prevented.
- Totals and rankings are calculated by hand and are available only after the event, with no live dashboard.
- There is no role-based access, so every judge or helper who has the spreadsheet can see and edit all the data.
- Notifications and reminders depend on the organiser remembering to send them.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	6 to 8 weeks (estimate) for database design, back end, front end, authentication, reports and charts	About 2 weeks (estimate) using forms, reports, pages and short scripts
Skills required	A web framework, a database, a front-end library and deployment skills	Understanding of data modelling and basic Deluge scripting
Hosting & infrastructure	Separate server, database and domain to be arranged and maintained	Built-in cloud hosting on the Zoho platform
Mobile application	Separate mobile or responsive front end to be coded	Responsive web interface provided by the platform (mobile app not used here)
Security & access control	Login, sessions and permissions coded by the developer	Built-in users, roles and permission sets configured through the UI
Maintenance & change	Code changes, testing and redeployment for every rule change	Rule changes made by editing a form or a script in the builder
Customisation limits	Unlimited, at the cost of effort	Limited to what the platform exposes; pixel-level UI and heavy computation are constrained
Cost	Developer time plus hosting charges	Free trial edition for development; paid editions needed for larger use

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Sanchis, García-Perales, Fraile and Poler, 2020 [5]	Low-Code as Enabler of Digital Transformation in Manufacturing Industry	Reports that low-code platforms shorten development and let business users build applications, and also discusses challenges such as governance and platform dependence	Supports the choice of low-code and the governance controls described in Chapter 7

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
2	Sahay, Indamutsa, Di Ruscio and Pierantonio, 2020 [6]	Supporting the Understanding and Comparison of Low-Code Development Platforms	Proposes a feature-based way of comparing low-code platforms, showing that platforms differ widely in modelling, automation and integration support	Guided the feature checklist in Appendix A
3	Di Ruscio, Kolovos, de Lara, Pierantonio, Tisi and Wimmer, 2022 [7]	Low-Code Development and Model-Driven Engineering: Two Sides of the Same Coin?	Relates low-code platforms to model-driven engineering and discusses open challenges	Explains the data-model-first design used in Chapter 4
4	Devpost, 2026 [8]	Devpost hackathon management platform	Commercial online platform used to run hackathons, covering project submission and judging	Shows that the problem is real; the proposed system is a free, customisable college-level alternative
5	Zoho Corporation [9]	Deluge Script Reference Guide	Documents the Deluge language: fetch, conditions, sendmail, alerts and workflow events	Basis of the seven scripts in Chapter 5
6	Zoho Corporation [10]	Zoho Creator Learning Centre	Learning material on forms, reports, pages, workflows and AI fields in Zoho Creator	Basis of the application design and the AI field

2.5 Emerging Trends in Low-Code Development

Three trends in low-code development influenced the design [3], [7]. First, AI-assisted features are now part of the platforms; the Keyword Extraction field used on the Team form is an example of an AI capability that can be added without writing a model. Second, citizen development means that organisers who are not professional programmers can maintain the application, so rules were kept in short, readable scripts. Third, hyper-automation, the chaining of validations, notifications and scheduled jobs, motivated the use of form workflows, a schedule and an approval process instead of manual follow-up.

2.6 Proposed System

The proposed system is a Zoho Creator application with four connected forms, five reports and a dashboard. It removes duplicate data entry by using lookups instead of re-typing team and judge names, removes manual totalling with a formula field, and replaces eye-checking with validation scripts that reject out-of-range scores, duplicate teams and repeated judging. E-mail alerts and a weekly schedule replace manual reminders, the approval process gives team registration an auditable decision, and the permission sets restrict what each role can open. The leaderboard and dashboard charts are available immediately after each score is saved.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator (Organiser)	Owner of the application and member of the organising committee; mapped to the CEO role of the role hierarchy	Create and manage participants, judges and teams; approve teams; review all reports and the dashboard; receive the e-mail alerts
Judge	Faculty or industry evaluator invited to score teams; mapped to the Judge role and the Judge Access permission set	Open the Scoring form and submit marks for each team; view the Scoring Report, Leaderboard, All Teams report and dashboard
Participant	Student who registers for the hackathon and leads or joins a team; mapped to the Participant role and the Participant Access permission set	Register through the Participant form; view the All Teams report and the dashboard to follow team status and the leaderboard charts

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall register a participant with name, college, experience level, skill and a mandatory e-mail address.	Administrator, Participant	High
FR-02	The system shall register a judge with name, domain expertise and e-mail address.	Administrator	Medium
FR-03	The system shall record a team with its name, problem statement, status and a team leader chosen from registered participants.	Administrator, Participant	High
FR-04	The system shall reject a team whose name already exists.	Administrator, Participant	High
FR-05	The system shall route team registrations through an approval process before a team is treated as approved.	Administrator	High
FR-06	The system shall accept scores for Innovation, Technical, Presentation and Impact from a judge for a team and reject any value outside 0 to 10.	Judge	High

FR ID	Requirement	Role(s)	Priority
FR-07	The system shall compute the total score automatically as the sum of the four criteria.	Judge, Administrator	High
FR-08	The system shall reject a second score from the same judge for the same team.	Judge	High
FR-09	The system shall send an e-mail alert to the administrator when a participant registers, when a score is submitted and when a team is approved.	Administrator	Medium
FR-10	The system shall send a scheduled reminder e-mail asking judges to complete pending scoring.	Administrator, Judge	Medium
FR-11	The system shall display the scores of all teams in a Scoring Report and a Leaderboard ranked by total score.	Administrator, Judge	High
FR-12	The system shall display a dashboard with the number of teams by status and the total score of each team.	Administrator, Judge, Participant	Medium
FR-13	The system shall restrict each role to the components permitted to it by its permission set.	Administrator	High
FR-14	The system shall extract keywords from a team's problem statement using the Keyword Extraction AI field.	Administrator, Judge	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard shall open within 5 seconds for the data volume of a college hackathon (up to about 100 records per form).
NFR-02	Security	Users shall open only the forms, reports and pages allowed by their permission set; automatic e-mails shall be addressed only to the application administrator.
NFR-03	Usability	The application shall be usable in a standard web browser without installation, with the menu grouped by Participants, Teams, Judges, Scoring and Dashboard.
NFR-04	Availability	The application shall be available whenever the Zoho Creator service is available, since it is hosted on the platform and needs no separate server.
NFR-05	Maintainability	Every business rule shall be implemented as a short, named workflow script so that the organiser can change it from the application builder.
NFR-06	Data Integrity	The system shall prevent duplicate teams, duplicate judge–team scores and scores outside the allowed range at the time of data entry.
NFR-07	Auditability	Workflows shall be named by purpose and kept enabled, and sample data shall use only reserved example e-mail addresses so that no real person is contacted.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator, free trial edition (India data centre)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not used; the application makes no external REST API calls
Other Zoho services	Zoho Mail (Deluge sendmail task) and Zoho Zia Keyword Extraction (AI field)
Mobile	Not used; the application was developed and tested in the web interface
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

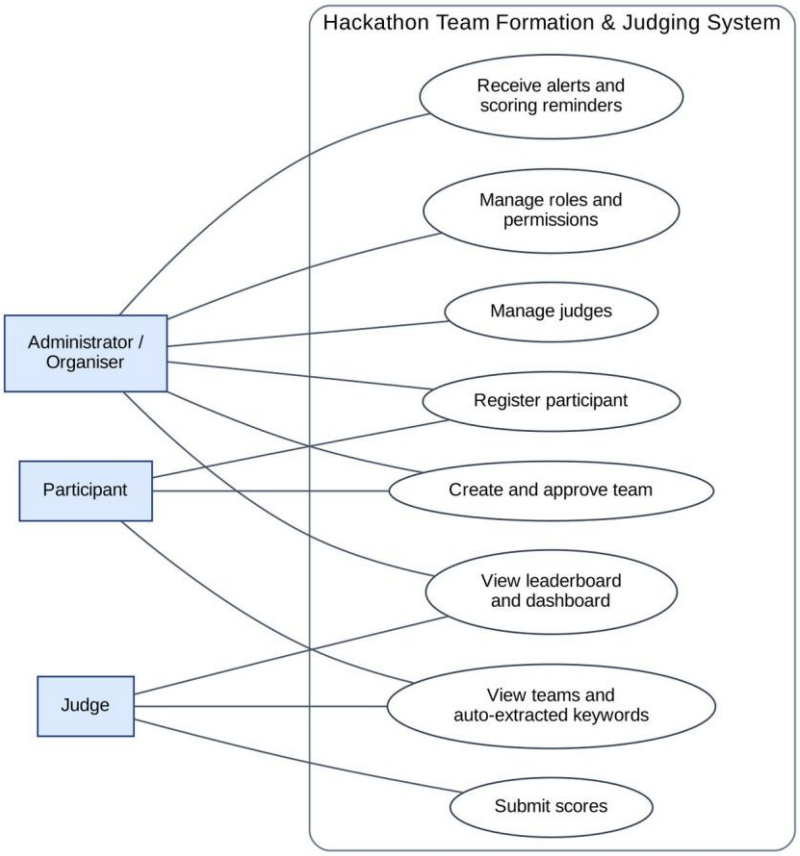


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Register Participant

Use Case ID	UC-01
Use Case Name	Register Participant
Actor(s)	Participant (administrator may also enter the record)
Pre-condition	The Participant form is open to the user through the Participant Access permission set.
Main Flow	1. The participant opens the Participant form. 2. Enters name, college, experience level, skill and e-mail. 3. Submits the form. 4. The record is saved and the success workflow sends a 'New participant registered' e-mail to the administrator.
Alternate Flow	The e-mail field is empty or not a valid e-mail address: the platform rejects the submission and the participant corrects it.
Post-condition	A Participant record exists and appears in the All Participants report.
Related FR	FR-01, FR-09, FR-13

Table 3.6: Use Case Description – Submit Team Score

Use Case ID	UC-02
Use Case Name	Submit Team Score
Actor(s)	Judge
Pre-condition	The judge has the Judge Access permission set, and the team and judge records exist.
Main Flow	1. The judge opens the Scoring form. 2. Selects the team and himself or herself in the Judge field. 3. Enters Innovation, Technical, Presentation and Impact marks. 4. Submits. 5. DS-01 and DS-02 validate the marks and the pair. 6. The record is saved, Total_Score is computed and DS-03 e-mails the administrator.
Alternate Flow	A mark above 10 or below 0, or a repeat score for the same team by the same judge, is rejected with an alert message and the form is not saved.
Post-condition	A Scoring record exists; the Scoring Report, Leaderboard and dashboard charts include it.
Related FR	FR-06, FR-07, FR-08, FR-09, FR-11

Table 3.7: Use Case Description – Approve Team

Use Case ID	UC-03
Use Case Name	Approve Team
Actor(s)	Administrator
Pre-condition	A team with status Pending has been registered.
Main Flow	1. The administrator opens the Team record from the All Teams report. 2. Reviews the problem statement and the keywords extracted by the AI field. 3. Sets the status to Approved. 4. DS-05 e-mails the administrator that the team is approved.
Alternate Flow	The administrator leaves the status as Pending; the team stays in the Pending bar of the dashboard chart.
Post-condition	The team is counted under Approved in the Teams by Status chart.
Related FR	FR-03, FR-05, FR-09, FR-12, FR-14

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

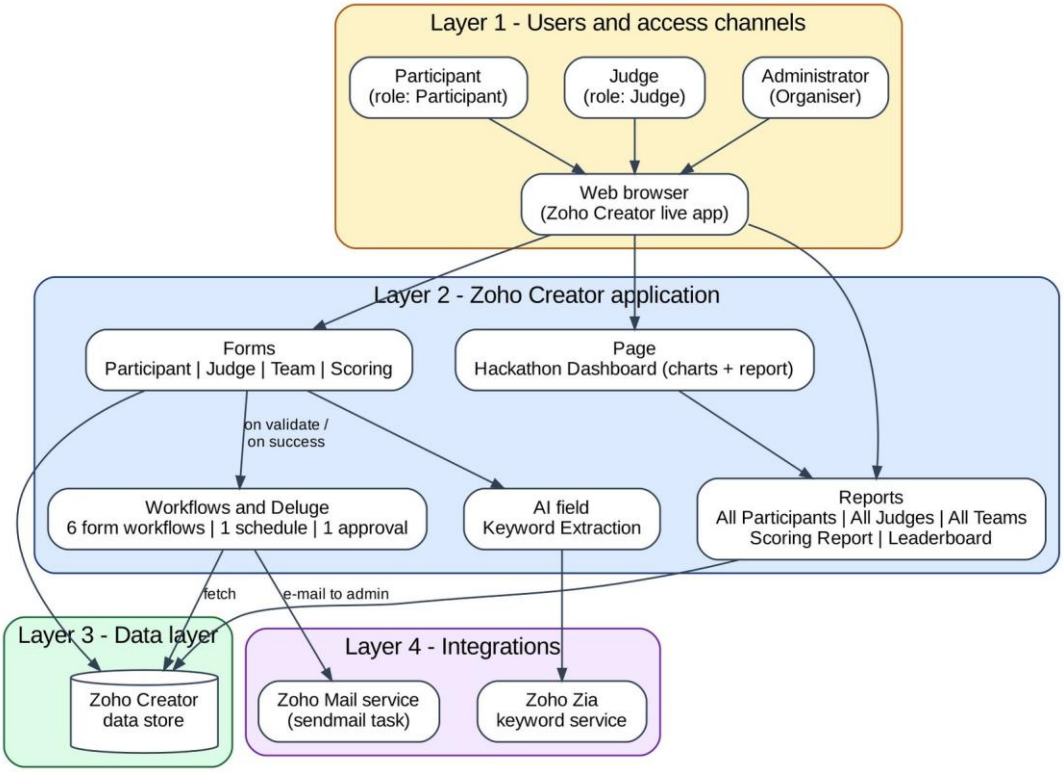


Figure 4.1: System Architecture of the Application

The application follows a layered architecture hosted entirely on the Zoho Creator platform. The access layer consists of the three roles (Administrator, Judge and Participant), who reach the application through a web browser; the application menu shown to each user depends on the permission set assigned to the role. The application layer holds the four forms, five reports, the dashboard page, the seven Deluge scripts and the approval process. The data layer is the platform's built-in cloud database, where the four forms are stored as related tables joined by lookup fields. The integration layer consists of Zoho Mail, which delivers the e-mails sent by the Deluge sendmail task, and the Zoho Zia Keyword Extraction service used by the AI field on the Team form. No external database, server or API was required.

4.2 Data Model

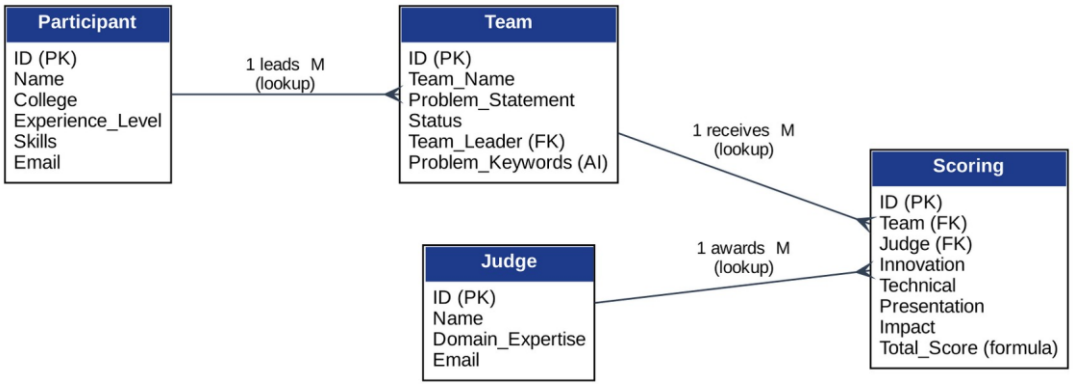


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Participant	Name, College, Experience_Level, Skills, Email	Team	One-to-many: one participant can be the leader of several teams	Lookup field Team_Leader in the Team form
Judge	Name, Domain_Expertise, Email	Scoring	One-to-many: one judge scores many teams	Lookup field Judge in the Scoring form
Team	Team_Name, Problem_Statement, Status, Team_Leader, Problem_Keywords	Participant, Scoring	Many-to-one with Participant; one-to-many with Scoring	Lookup fields Team_Leader and Scoring.Team
Scoring	Innovation, Technical, Presentation, Impact, Total_Score, Team, Judge	Team, Judge	Many-to-one with each; together they give a many-to-many relationship between Team and Judge	Lookup fields Team and Judge

Participant, Judge and Team hold the master data, and Scoring is the transaction form that links a judge to a team with four marks. A many-to-many relationship between teams and judges is needed because every team is scored by more than one judge and every judge scores more than one team; in Zoho Creator this is modelled by the Scoring form carrying two lookup fields rather than by a subform. A subform was not used

because a score belongs to the pair of a team and a judge and is entered by the judge, not by the team owner.

4.3 Form Design

Table 4.2: Field Specification – Participant Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Name	Name	Name	No	First and last name sub-fields	Composite field; the New Participant Alert script reads Name.first_name and Name.last_name
College	College	Single Line	No	None	Institution of the participant
Experience Level	Experience_Level	Drop Down	No	Choices: Beginner, Intermediate, Advanced	Used in the New Participant Alert e-mail
Skills	Skills	Drop Down	No	Choices: Web Development, Machine Learning, UI/UX Design, Backend / APIs, Data Analysis	Main skill of the participant
Email	Email	Email	Yes	Valid e-mail format checked by the platform	Used for identification; sample data use @example.com

Table 4.3: Field Specification – Judge Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Name	Name	Name	No	First and last name sub-fields	Composite field
Domain Expertise	Domain_Expertise	Single Line	No	None	Free-text area of expertise, e.g. Web Development
Email	Email	Email	Yes	Valid e-mail format checked by the platform	Printed in the Score Submitted Alert

Table 4.4: Field Specification – Team Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Team Name	Team_Name	Single Line	No	Duplicate names rejected by DS-04	Unique name of the team
Problem Statement	Problem_Statement	Multi Line	No	None	Source text for the AI keyword field
Status	Status	Drop Down	No	Choices: Pending, Approved	Approval and DS-05 depend on this field
Team Leader	Team_Leader	Lookup	No	Lookup to the Participant form	Relates a team to its leader
Problem Keywords	Problem_Keywords	Keyword Extraction (AI field)	No	Filled automatically from Problem_Statement	Zia AI field; see Section 5.9

Table 4.5: Field Specification – Scoring Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Team	Team	Lookup	No	Lookup to the Team form	One half of the many-to-many link
Judge	Judge	Lookup	No	Lookup to the Judge form; pair with Team checked by DS-02	Other half of the many-to-many link
Innovation	Innovation	Number	No	0 to 10, enforced by DS-01	Criterion 1
Technical	Technical	Number	No	0 to 10, enforced by DS-01	Criterion 2
Presentation	Presentation	Number	No	0 to 10, enforced by DS-01	Criterion 3
Impact	Impact	Number	No	0 to 10, enforced by DS-01	Criterion 4
Total Score	Total_Score	Formula	No (calculated)	Sum of the four criteria; maximum 40	Read-only; used by the report and charts

4.4 Process and Workflow Design

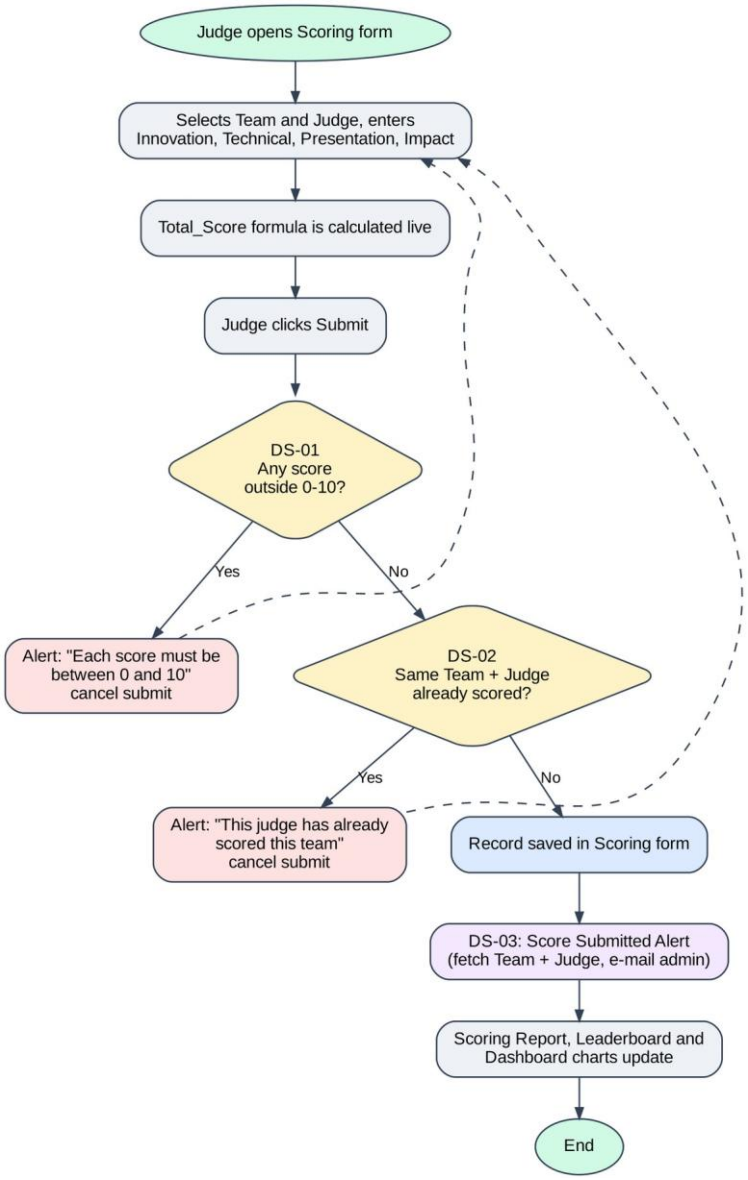


Figure 4.3: Workflow / Process Flow for Score Submission and Validation

Figure 4.3 shows the most rule-intensive process of the application. When a judge submits the Scoring form, the On Validate scripts DS-01 and DS-02 run first and cancel the submission with an alert if a mark is outside 0 to 10 or if the judge has already scored the team. Only a valid record is saved; the On Success script DS-03 then fetches the team and the judge and e-mails the administrator, and the reports and dashboard charts read the new record at once. The team registration process follows the same pattern with DS-04 for duplicate team names, the approval process for the decision and DS-05 for the approval alert.

4.5 User Interface and Navigation Design

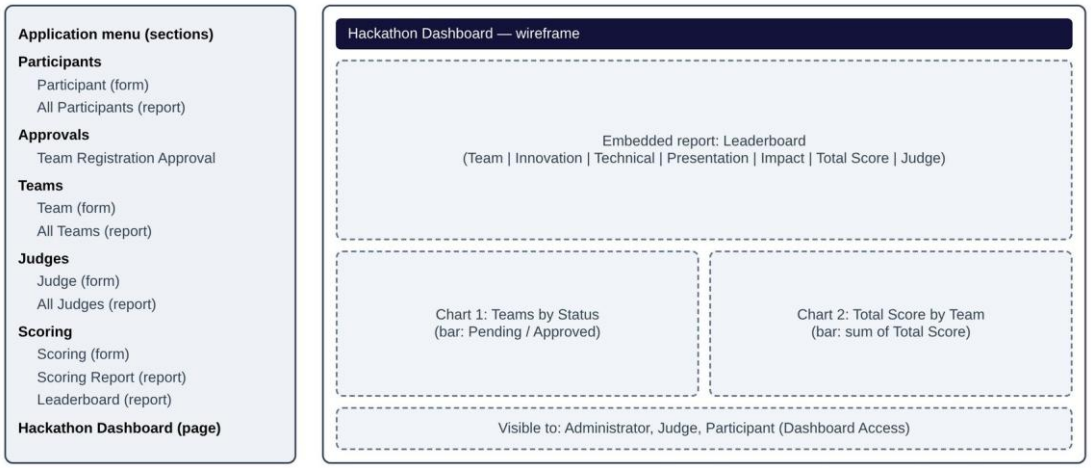


Figure 4.4: Navigation Structure and Wireframe of the Hackathon Dashboard Page

The menu is organised into five sections that follow the order of the event: Participants, Teams, Judges, Scoring and the Hackathon Dashboard, with an Approvals section for the team approval process. Each form is placed beside the report that lists its records, so a user who adds a record can see it immediately. The dashboard page was planned as a single screen with the leaderboard on top and two bar charts below it, so that a judge or participant sees the standing of the teams without opening any other report.

4.6 Access Control Design

Table 4.6: Role–Permission Matrix

Component	Administrator	Judge	Participant
Form: Participant	A E V D	—	Access
Form: Judge	A E V D	—	—
Form: Team	A E V D	Access	Access
Form: Scoring	A E V D	Access	—
Report: All Participants, All Judges	V	—	—
Report: All Teams	V	V	V
Report: Scoring Report, Leaderboard	V	V	—
Page: Hackathon Dashboard	V	V	V

A = Add, E = Edit, V = View, D = Delete, — = No access. ‘Access’ means that the component is available to the role through its permission set; the Administrator, as owner of the application, has full access to every component.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created from scratch in the Zoho Creator workspace of the owner account, in the India data centre, using the application name Hackathon Team Formation & Judging System (link name hackathon-team-formation-judging-system). No template, import or AI-assisted app builder was used; the four forms were designed one by one in the form builder. The menu was organised into sections that follow the flow of the event: Participants, Approvals, Teams, Judges, Scoring and the Hackathon Dashboard page, and each form was placed beside the report that lists its records. The default application theme was kept, because the colours and layout of the platform are already readable and the effort of the project was directed at data, rules and automation.

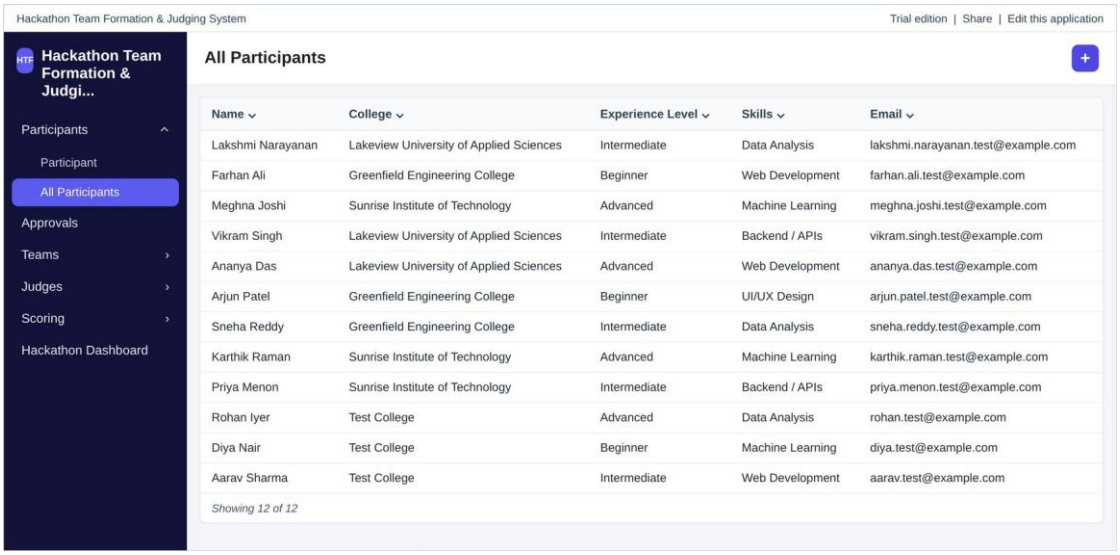


Figure 5.1: Application Home and Navigation Sections (All Participants Report)

5.2 Forms and Data Capture

Four forms were built. Participant stores the name (Name field with first and last name), college (Single Line), experience level and main skill (Drop Down fields) and a mandatory e-mail address. Judge stores the name, domain expertise (Single Line) and e-mail address. Team stores the team name, a Multi Line problem statement, a Drop Down status (Pending or Approved), a Lookup to the Participant form for the team leader and a Keyword Extraction AI field that is filled from the problem statement. Scoring stores four Number fields for Innovation, Technical, Presentation and Impact, a

Formula field Total_Score that adds the four criteria, and two Lookup fields to the Team and Judge forms. Field-level checks were kept in Deluge on validation, so that the rules are visible in one place, and the success message and submit button of each form were left at their defaults. Table 4.2 to Table 4.5 give the field specification, and Figure 5.2 and Figure 5.3 show a form in the builder and in live mode.

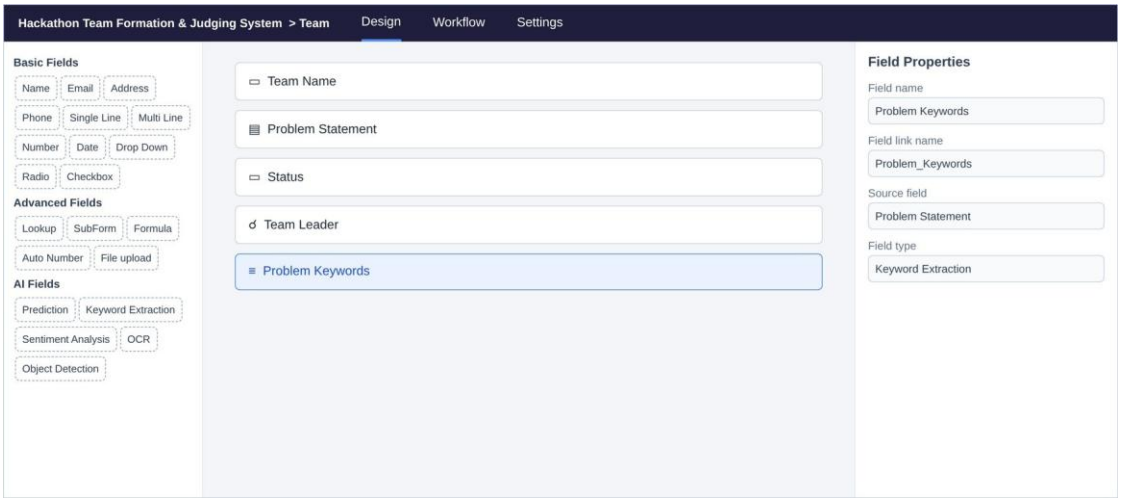


Figure 5.2: Team Form with the Problem Keywords AI Field in the Form Builder

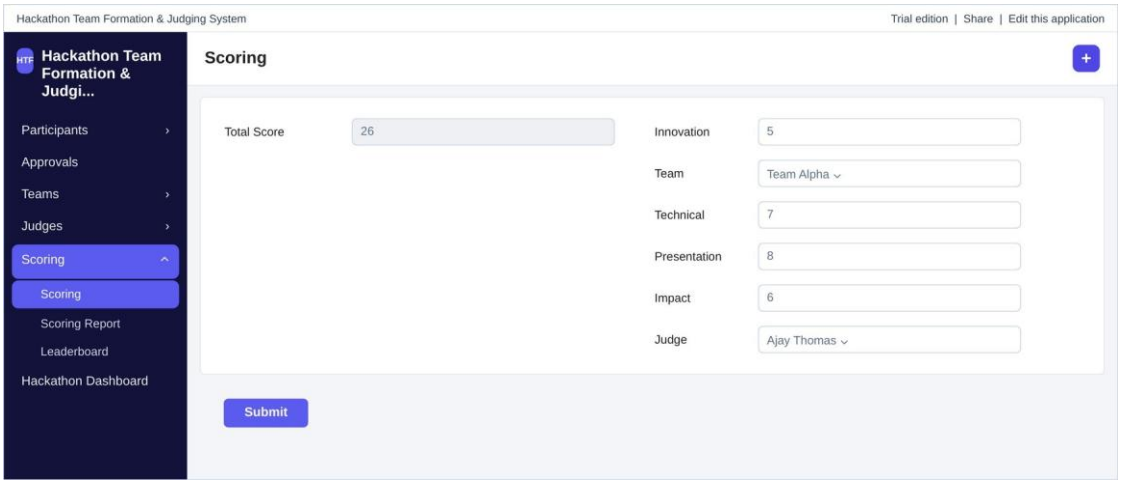


Figure 5.3: Scoring Form in Live Mode

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Participants	List	Participant	All records; default sorting	Default record actions	Administrator

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Judges	List	Judge	All records; default sorting	Default record actions	Administrator
All Teams	List	Team	All records; shows status, team leader and AI keywords	Default record actions	Administrator, Judge, Participant
Scoring Report	List	Scoring	All score records with the four criteria, Total_Score, team and judge	Default record actions	Administrator, Judge
Leaderboard	List	Scoring	Sorted by Total_Score from highest to lowest	Default record actions	Administrator, Judge

Five list reports were created, one for each form and a second report on the Scoring form that is sorted by total score. The reports were kept as list reports because the data is tabular and small; Kanban, calendar or map reports were considered but have no natural date, stage or location field in this data model. Judges and participants see only the reports that their permission set allows.

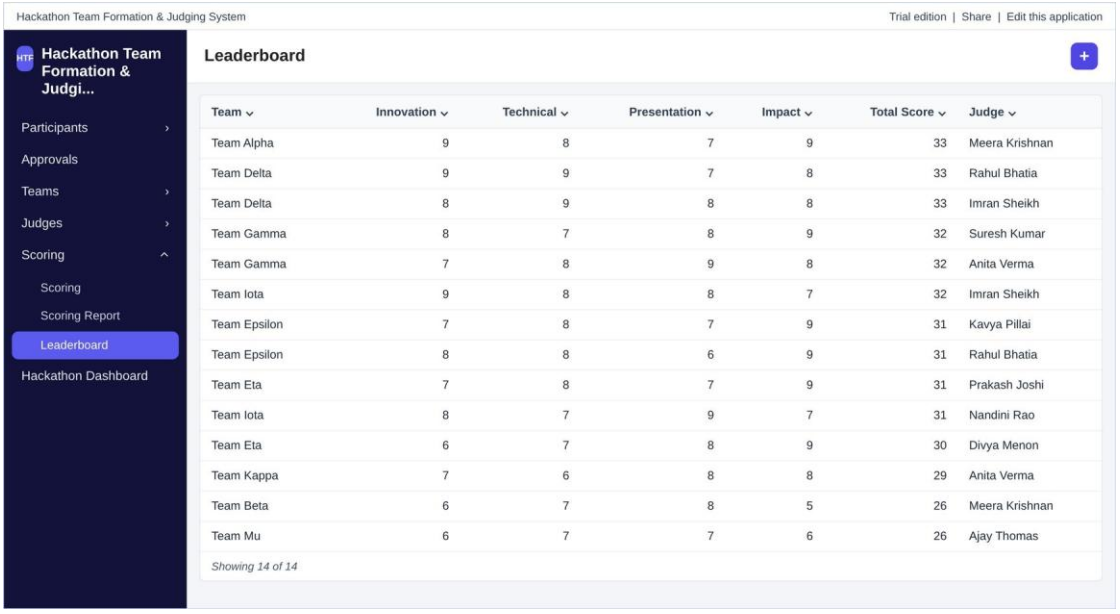


Figure 5.4: Leaderboard Report

5.4 Pages and Dashboards

One page, Hackathon Dashboard, was built with the page builder. It shows the Leaderboard report at the top and two bar charts below it: Teams by Status (the number of teams that are Pending and Approved) and Total Score by Team (the sum of Total_Score of every team). The page is available to all three roles through the

Dashboard permission, so that a judge sees the progress of scoring and a participant sees the standing of the teams without opening each report. On the seed data the status chart shows 4 Pending and 8 Approved teams, and the total-score chart shows Team Delta highest with 66 points from two judges.

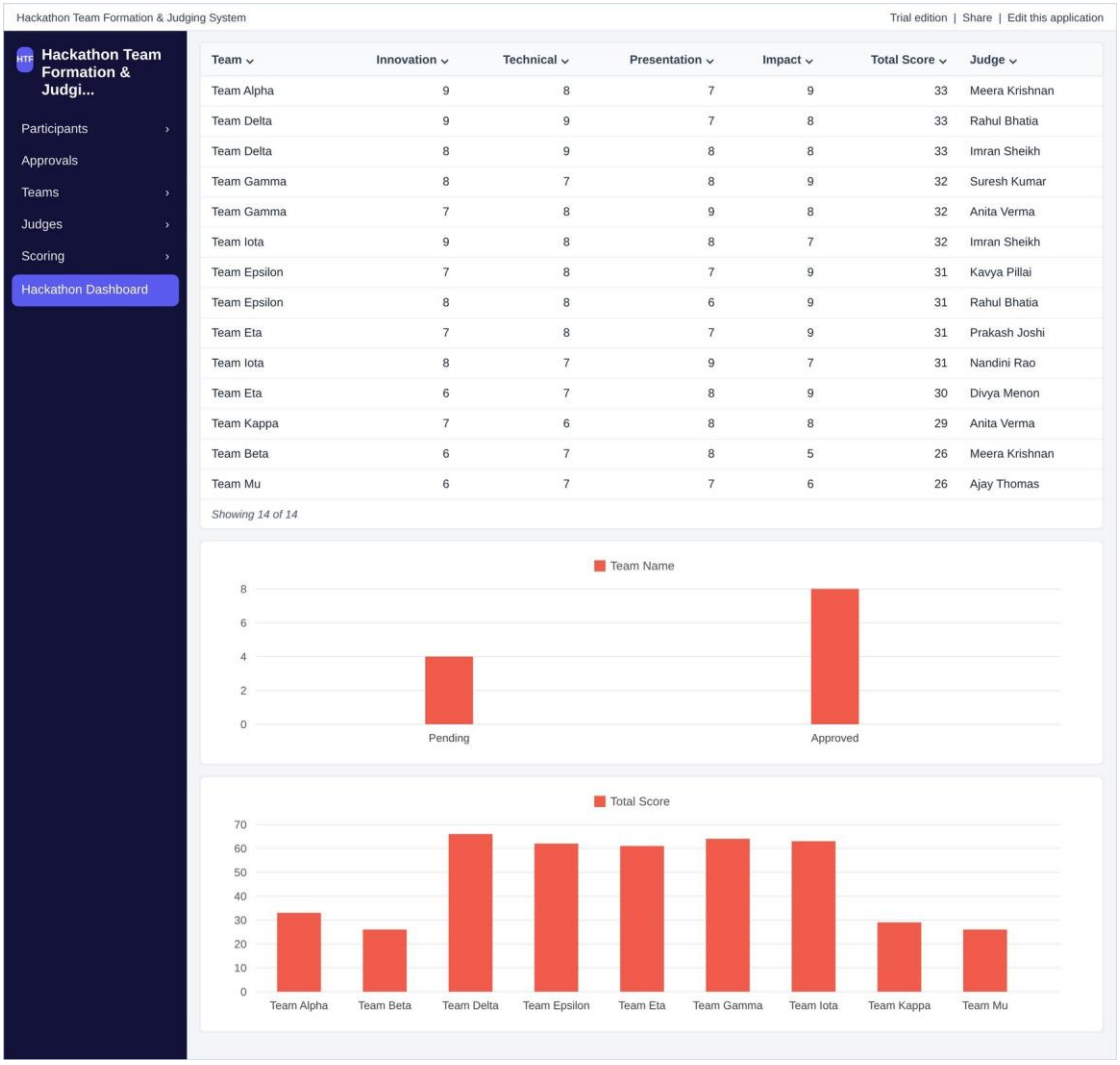


Figure 5.5: Hackathon Dashboard Page

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Scoring form → On Validate (Validate Score Range)	Reject a score below 0 or above 10 in any of the four criteria	if condition, comparison and logical OR operators, alert, cancel submit
DS-02	Scoring form → On Validate (Prevent Duplicate Scoring)	Stop the same judge scoring the same team twice	Criteria fetch with .count(), null check, alert, cancel submit

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-03	Scoring form → On Success (Score Submitted Alert)	E-mail the administrator the team, judge and total (out of 40)	Fetch by ID, ifnull, string concatenation, sendmail, info
DS-04	Team form → On Validate (Prevent Duplicate Team)	Reject a team whose name already exists	Criteria fetch with .count(), null check, alert, cancel submit
DS-05	Team form → On Edit, Success (Notify Team Approved)	E-mail the administrator when a team's status is Approved	if condition, string comparison, sendmail
DS-06	Participant form → On Success (New Participant Alert)	E-mail the administrator the details of a new participant	Composite Name sub-fields, string concatenation, sendmail, info
DS-07	Custom schedule (Judge Scoring Reminder)	Weekly reminder to judges to finish scoring, sent to the administrator	Schedule trigger, sendmail

Code 5.1: Duplicate-Scoring Validation Script (DS-02)

```
// Script ID : DS-02
// Location  : Scoring → On Validate (workflow: Prevent Duplicate
Scoring)
// Purpose   : Stop a judge from scoring the same team twice
if(input.Team != null && input.Judge != null && Scoring[Team ==
input.Team && Judge == input.Judge].count() > 0)
{
    alert "This judge has already scored this team.";
    cancel submit;
}
```

Code 5.2: Score Notification Script (DS-03)

```
// Script ID : DS-03
// Location  : Scoring → On Success (workflow: Score Submitted Alert)
// Purpose   : E-mail the administrator when a score is saved
t = Team[ID == input.Team];
j = Judge[ID == input.Judge];
total = ifnull(input.Innovation,0) + ifnull(input.Technical,0) +
ifnull(input.Presentation,0) + ifnull(input.Impact,0);
sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"New score submitted for " + t.Team_Name
    message :"A score was submitted for team " + t.Team_Name + " by
judge " + j.Email + ". Total score: " + total + " out of 40."
]
info "Score alert sent for " + t.Team_Name;
```

DS-01 and DS-02 run on the On Validate event of the Scoring form, before the record is saved. DS-01 compares each of the four marks with 0 and 10; DS-02 fetches the Scoring

records with the same team and judge and counts them, using a criteria fetch so that only matching records are read. If either condition is true the script shows an alert and cancels the submission. DS-04 follows the same pattern for the team name. When testing, a score of 11 and a second score by the same judge for the same team were both blocked with the alert message, and no record was created.

DS-03 runs after a valid Scoring record is saved. It fetches the team and the judge by the IDs stored in the lookup fields, adds the four marks with ifnull so that an empty mark counts as zero, and builds the e-mail text. The mail is addressed from and to zoho.adminuserid, which is the application administrator, so no judge or participant is ever contacted. The info statement writes a line to the log that was used to confirm that the script had run. DS-05, DS-06 and DS-07 are shorter alert scripts and their full text is given in Appendix C.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Validate Score Range	Form workflow	Scoring: created, on validate	Any mark below 0 or above 10	Alert and cancel submit	DS-01
Prevent Duplicate Scoring	Form workflow	Scoring: created, on validate	Same team and judge already scored	Alert and cancel submit	DS-02
Score Submitted Alert	Form workflow	Scoring: created, successful submission	None (always)	E-mail to administrator	DS-03
Prevent Duplicate Team	Form workflow	Team: created, on validate	A team with the same name exists	Alert and cancel submit	DS-04
Notify Team Approved	Form workflow	Team: edited, successful submission	Status is Approved	E-mail to administrator	DS-05
New Participant Alert	Form workflow	Participant: created, successful submission	None (always)	E-mail to administrator	DS-06
Judge Scoring Reminder	Schedule	Custom schedule; first run 18-Oct-2026 at 20:00, repeating weekly	None (always)	Reminder e-mail to administrator	DS-07

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Team Registration Approval	Approval process	Team registration submitted for approval	Decision of the approver	Approve or reject the team registration	—

Six form workflows (all enabled), one schedule (enabled) and one approval process are configured. A Blueprint, a batch workflow and a payment workflow were not used: the process has only two statuses (Pending and Approved), so the approval process is sufficient and a Blueprint would add stages without adding a rule; and there is no bulk data operation that needs a batch workflow.



Figure 5.6: Score Submitted Alert Workflow Configuration

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho Mail	Built-in Zoho Mail service of the platform	Deluge sendmail task (from and to zoho.adminuserid)	No credentials stored in the application; mail is sent as the application administrator	Alert and reminder e-mails to the administrator (DS-03, DS-05, DS-06, DS-07)
Zoho Zia Keyword Extraction	Built-in AI field type of Zoho Creator	Keyword Extraction field with Problem Statement as source field	Handled by the platform	Problem statement text is analysed and keywords are stored in Problem_Keywords

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
External REST API, webhook, Zoho CRM	Not implemented	—	—	Out of scope; see Section 5.10 and Chapter 9

Code 5.3: Team-Approval Notification Script Using Zoho Mail (DS-05)

```
// Script ID : DS-05
// Location  : Team → On Edit, successful submission (workflow: Notify
Team Approved)
// Purpose   : E-mail the administrator when a team is set to Approved
if(input.Status == "Approved")
{
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"Team Approved: " + input.Team_Name
        message :"Team " + input.Team_Name + " has been approved for
the hackathon."
    ]
}
```

Integration in this application is limited to Zoho's own services, which need no connection setup, API key or token. The sendmail task in DS-05 (Code 5.3) is the pattern used by all four alert scripts: the sender and the recipient are both the administrator, and the subject and message are built from the record that triggered the script. Figure 5.7 shows the path of an alert from the trigger to the administrator's mailbox. Because no external API is called, Postman was not needed, and the arrangement avoids any stored secret.

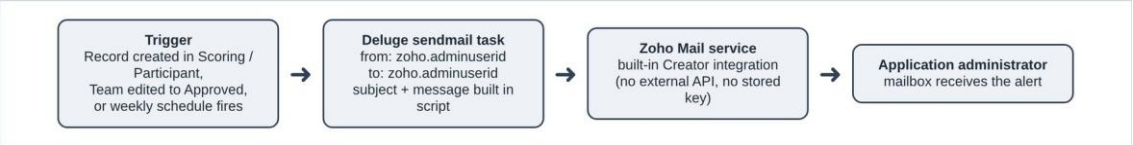


Figure 5.7: E-mail Integration Flow from Trigger to Administrator

5.8 User Management and Role-Based Access

Access control was implemented with a role hierarchy and two permission sets. Two roles, Judge and Participant, were created below the CEO role, which belongs to the owner and keeps full access as administrator. The permission set Judge Access gives access to the Scoring form with two reports (Scoring Report and Leaderboard), the Team form with the All Teams report, and the dashboard. The permission set Participant Access gives access to the Participant form, the All Teams report and the dashboard. Record-level criteria were not required because all data in the event is visible to the people who hold the role. No real users were invited to the application; the roles and permission sets were configured and checked in the settings, so role-based

login tests were not run with separate user accounts. This matches the matrix in Table 4.6.

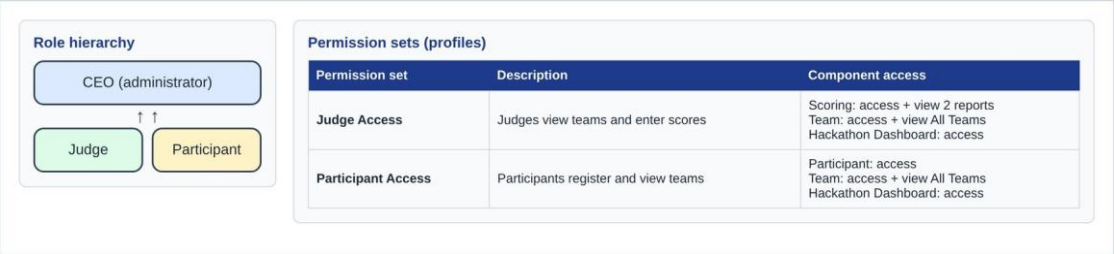


Figure 5.8: Roles and Permission Sets Configuration

5.9 AI and Analytics Features

One Zoho Zia feature was used: the Keyword Extraction AI field Problem Keywords on the Team form, whose source is the Problem Statement field. When a team is saved, the platform returns the key phrases of the problem statement, for example ‘telemedicine app, clinics’ for a rural-clinic project and ‘face detection, exam proctoring’ for the online proctoring project (Figure 5.9). The keywords were checked by reading them against the statements. The extraction is imperfect: short statements such as ‘Hostel mess food waste tracker’ and ‘Smart campus attendance tracker’ returned no keywords, and some phrases are cut (for example ‘-time availability’), so the field is treated as a convenience for grouping teams and not as a rule input.

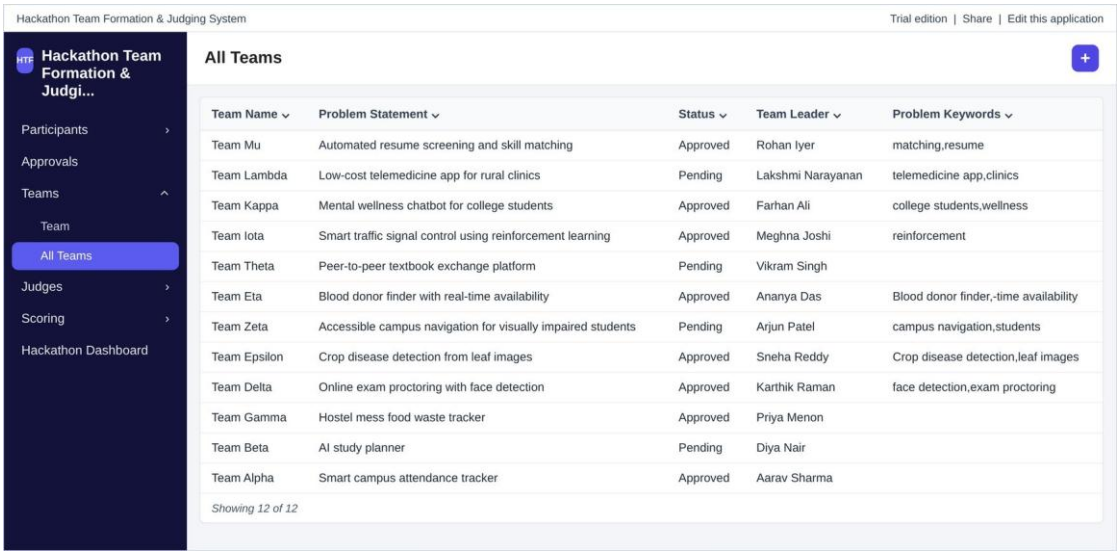


Figure 5.9: AI Keyword Extraction in the All Teams Report

5.10 Pro-Code Extension

No pro-code extension was built for this application, because every requirement of the event could be met with forms, workflows and Deluge. Zoho Catalyst, JavaScript widgets or the Creator REST API would be justified if the event needed a heavy computation such as plagiarism checking of submitted projects, a custom public leaderboard website or a real-time scoreboard on a projector; in that case a Catalyst function could compute the ranking and a widget could display it inside the Creator page.

5.11 Mobile and Portal Access

The Zoho Creator mobile application and the customer portal were not configured for this project. The application was developed and tested in the web interface only, so no mobile figure is included.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was carried out at four levels on the live application. Unit testing covered each form and each Deluge script on its own: valid data was entered, and then invalid data was entered to confirm that the validation scripts reject it. Integration testing followed one record through the chain of form, workflow, report and dashboard, for example a score from the Scoring form to the Leaderboard and the chart. The sample data (12 participants, 10 judges, 12 teams and 14 scores, all with @example.com e-mail addresses) were loaded by a temporary Deluge script that was deleted afterwards; the rules of the forms were then tested through the forms in live mode, and four negative tests were run in this way. Totals were compared with hand calculation for all 14 score records. Role-based testing was limited to checking the permission sets in the settings, because no real users were invited. User acceptance testing with outside users was not carried out and is therefore not reported.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Register a participant with valid data	Enter name, college, level, skill and e-mail; submit	Record saved and listed	Record saved; 12 participants listed in All Participants	Pass
TC-02	FR-01	Submit the Participant form without e-mail (negative)	Leave Email empty; submit	Form rejected with a required-field message	Rejected; no record created	Pass
TC-03	FR-02	Add a judge	Enter name, domain expertise and e-mail	Record saved and listed	Saved; 10 judges listed in All Judges	Pass
TC-04	FR-03	Add a team with a leader	Enter team name, problem statement, status; choose a participant as leader	Team saved with the leader's name	Saved; 12 teams with leaders listed in All Teams	Pass
TC-05	FR-04	Add a team with an existing name (negative)	Enter 'Team Alpha' again; submit	Alert and no new record (DS-04)	Alert shown; no record created	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-06	FR-05	Route a team through the approval process	Open Approvals; process a Pending team	Team approved or rejected by the approver	Process is configured under Approvals; an end-to-end run by a second approver was not executed	Partial
TC-07	FR-06	Score above the range (negative)	Enter Innovation = 11; submit	Alert 'Each score must be between 0 and 10' (DS-01)	Alert shown; no record created	Pass
TC-08	FR-06	Enter valid scores	Choose team and judge; enter 9, 8, 7, 9	Record saved	Saved; appears in Scoring Report	Pass
TC-09	FR-07	Check the computed total	Compare Total_Score with the sum of four marks for all 14 records	Total equals the sum	All 14 totals matched (e.g. 9+8+7+9 = 33)	Pass
TC-10	FR-08	Score the same team twice by the same judge (negative)	Submit a second record for the same team and judge	Alert 'This judge has already scored this team' (DS-02)	Alert shown; no record created	Pass
TC-11	FR-09	Alert e-mails to the administrator	Register a participant, submit a score, approve a team	Three e-mails to the administrator	Scripts DS-03, DS-05 and DS-06 are saved and enabled; receipt of each mail was not individually confirmed	Partial
TC-12	FR-10	Weekly judge reminder	Check the schedule list	Schedule enabled and repeating	Enabled; first run 18-Oct-2026 20:00; receipt of a reminder not individually confirmed	Partial
TC-13	FR-11	Leaderboard ranking	Open Leaderboard	Records sorted by Total_Score, highest first	14 records in descending order of total	Pass
TC-14	FR-12	Dashboard charts	Compare charts with records	4 Pending and 8 Approved teams; Team Delta total 66	Charts agreed with the records	Pass
TC-15	FR-13	Permission sets	Inspect Judge Access and Participant Access	Components limited as in Table 4.6	Configuration matches the matrix; login as each role was not run	Partial

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-16	FR-14	AI keyword extraction	Save teams with problem statements	Keywords stored in Problem_Keywords	Keywords returned for 8 of 12 teams; none for 4 short statements	Pass

6.3 Debugging Deluge Scripts

Three techniques were used. First, the Deluge editor reports the line number and the reason when a script fails to save or run; this was the main tool. Second, info statements were placed at the end of the alert scripts (for example 'Score alert sent for <team>'), so that the execution log shows that a script ran to the end. Third, the alert messages of the validation scripts show the rule that failed on the form itself. One real bug was found while loading the sample data: the temporary insert script assigned the composite Name field as a map, and the platform returned the error 'You cannot assign 'MAP' value to Name Field' at line 9 (Figure 6.1). The cause was that Name is a composite field with first_name and last_name sub-fields. A second attempt with a plain string gave the same type of error. The fix was to assign Name.first_name and Name.last_name separately, after which all records were inserted.



Figure 6.1: Deluge Error for the Composite Name Field

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Insert of a participant failed with 'You cannot assign 'MAP' value to Name Field'	Name is a composite field; a map or a string cannot be assigned to it directly	Assigned Name.first_name and Name.last_name separately	Closed

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-02	Two workflows were created with the same name while building the Scoring and Team workflows	Names were repeated when similar workflows were created in sequence	Workflows were renamed by purpose (e.g. Validate Score Range, Prevent Duplicate Scoring)	Closed
BUG-03	The event setting of a Team workflow went back to its default after saving	The setting was not kept at the first save in the builder	Event was set again and the saved workflow was reopened to confirm it	Closed
BUG-04	A permission set did not keep a component selection at the first attempt	The check boxes of the permission set were not retained after the first save	Selections were repeated and the permission set was reopened to confirm them	Closed

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Separate Development, Stage and Production environments were not configured for this project: the application was built, tested and demonstrated in a single workspace of the free trial account. The release was therefore managed by discipline instead of by environment, as in Figure 7.1: the application was built in the builder, tested in live mode, released by using the application URL for the demonstration, and maintained by editing the application in the builder. The sample data were kept and were labelled by using only @example.com addresses so that they can be recognised. The rollback plan is simple: a workflow can be disabled or edited in the builder without affecting the data. A release note was not kept, which is listed as a gap in Table 7.1.

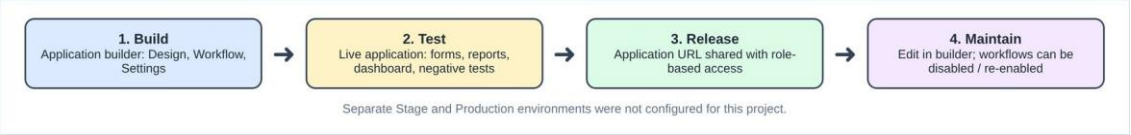


Figure 7.1: Build, Test, Release and Maintain Cycle Used in Place of Separate Environments

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓ (single workspace)
2	Sample/test data removed or isolated	✗ (kept for demonstration; @example.com only)
3	Roles, permissions and users configured in Production	✓ (roles and permission sets; no users invited)
4	Connections authorised in Production	N/A (no connections used)
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓ (application opens at its URL)

S.No.	Checklist Item	Done (✓)
7	Version / release notes recorded	✗ (not recorded)

7.2 Performance Optimisation

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Criteria fetch with .count() so that only matching records are examined	DS-02, DS-04	Reading every record and comparing in a loop (alternative design, not built)	Database filters on team and judge, or on team name	Not measurable at 14 to 12 records; the saving grows with data size
Fetch of one Team and one Judge by ID instead of scanning the forms	DS-03	Looping over the Team and Judge forms (alternative design, not built)	Two single-record fetches	Not measurable at this volume; constant effort per submission
Separate Leaderboard report sorted by Total_Score, in addition to the full Scoring Report	Leaderboard report	One report to be sorted by hand	Ranked list opens ready to read	Removes a manual step; load time not measured

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role and permission sets (Judge Access, Participant Access)	Settings → Roles and Permission Sets	Unauthorised access to forms, reports and pages
E-mails addressed only to zoho.adminuserid	DS-03, DS-05, DS-06, DS-07	Mail reaching real participants or judges by mistake during testing
Mandatory, validated Email field	Participant and Judge forms	Invalid contact data
Score range and duplicate checks	DS-01, DS-02, DS-04	Tampered or accidental bad data
No credentials or API keys in scripts	All scripts	Credential leakage
Reserved @example.com addresses in sample data	All sample records	Exposure or contact of real people

7.4 Governance and Administration

Governance in this project is light because there is one developer and one application. Workflows are named by purpose, for example Prevent Duplicate Team and Notify Team Approved, and every workflow is listed with its creation date in the workflow list, which serves as a simple change history. Script identifiers DS-01 to DS-07 are used in the report and in the script headers so that a script can be traced from the report to the builder. Responsibility for maintenance lies with the application owner, who is also the administrator. The trial account has an expiry date; before it ends the application would need a paid edition or the data would have to be exported from the reports. An audit trail screen and a data export were not used in this project, so Figure 7.2 summarises the governance controls that were applied instead of showing a built-in audit screen.



Figure 7.2: Governance Controls Applied to the Application

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario follows one hackathon through the three roles. The organiser registers the participants, who form twelve teams of different problem domains; ten judges from five domains are recorded; the judges score the teams, and the organiser follows the standing of the teams on the dashboard. At every step, the rules of Chapter 5 run in the background: a duplicate team name is refused, marks outside 0 to 10 are refused, a judge cannot score a team twice, and the administrator is informed by e-mail.

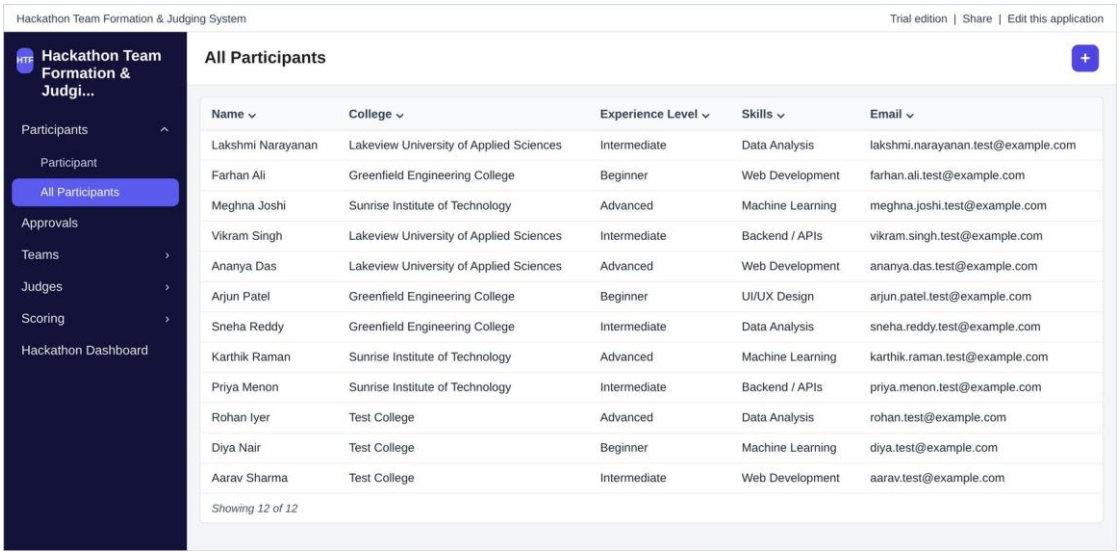


Figure 8.1: Scenario Step 1 – Participants Registered in the All Participants Report

Step 1. Twelve participants from different colleges are registered with their experience levels and skills. Each registration is e-mailed to the administrator by DS-06, and the e-mail field is mandatory.

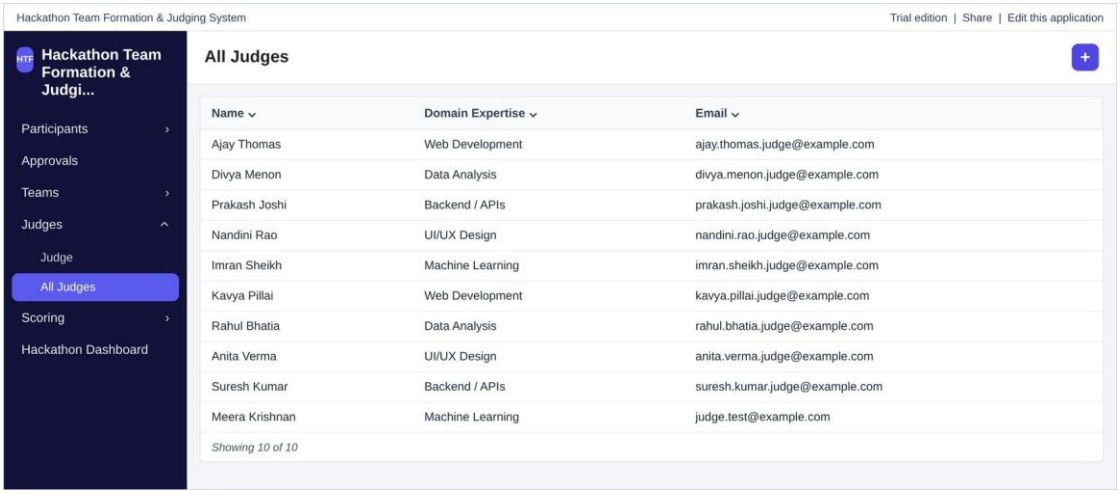


Figure 8.2: Scenario Step 2 – Judges Recorded in the All Judges Report

Step 2. Ten judges are recorded with their domain expertise: Web Development, Data Analysis, Backend / APIs, UI/UX Design and Machine Learning. Teams are created with a leader and a problem statement; eight teams are Approved and four are Pending, and the AI field adds keywords to the problem statements.

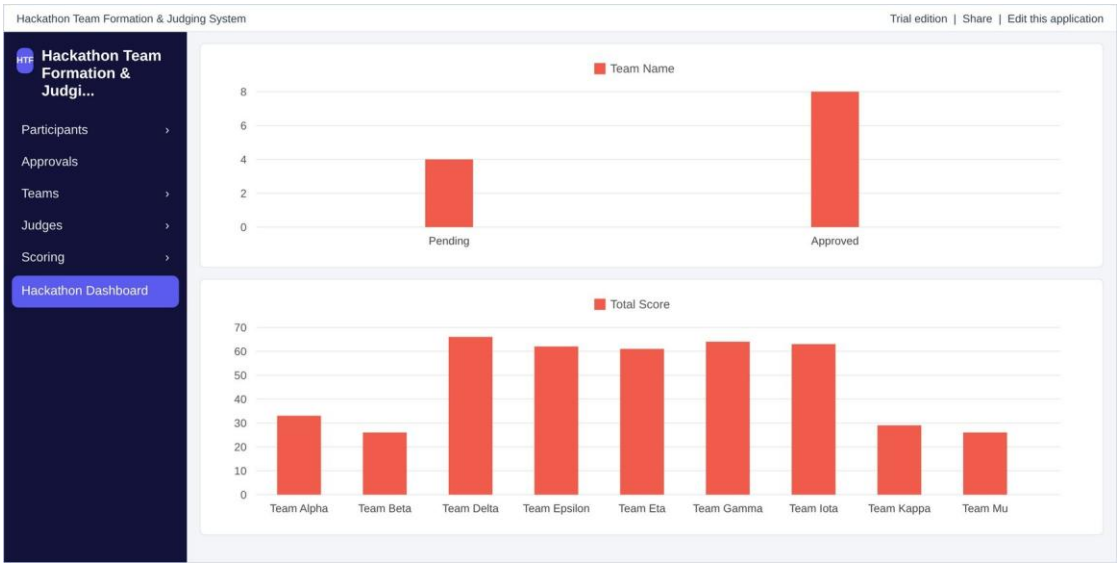


Figure 8.3: Scenario Step 3 – Dashboard Charts After Scoring

Step 3. Fourteen scores are entered. The Leaderboard ranks them by total, and the dashboard charts show 4 Pending and 8 Approved teams and the total of each team, with Team Delta (66 points from two judges) at the top.

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
To design a normalised data model and forms for participants, judges, teams and scores, connected by lookup fields	Sections 4.2, 4.3 and 5.2: four forms with lookups	Met: Table 4.1 and Figure 4.2; 12, 10, 12 and 14 records stored
To automate score validation, duplicate prevention, e-mail alerts and judge reminders using Deluge scripting and workflows	Sections 5.5 and 5.6: seven scripts DS-01 to DS-07	Met: four negative tests were rejected correctly (Table 6.1); alert delivery partly confirmed
To provide role-based access, dashboards and reports for the organiser, judges and participants	Sections 5.3, 5.4 and 5.8: five reports, one dashboard, two permission sets	Met in configuration; role-by-role login tests were not run
To integrate the application with Zoho Mail and Zoho Zia (Keyword Extraction)	Sections 5.7 and 5.9	Met: alerts through sendmail; keywords returned for 8 of 12 teams
To test the application with realistic data and document its security, deployment and governance controls	Chapters 6 and 7	Partly met: Stage and Production environments and user acceptance testing were not done

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Data entry	Registration, team list, score sheet and result sheet kept separately	One set of four connected forms	Each fact is entered once and reused through lookups
Score totalling	Hand or calculator totals	Formula field Total_Score	No arithmetic by hand; all 14 totals matched
Rule checking	Eye-checking for duplicates and invalid marks	Scripts DS-01, DS-02 and DS-04 on validation	Invalid entries are blocked at the time of entry
Result publication	Prepared after the event	Leaderboard and dashboard update after each saved score	Standing is visible during the event

Parameter	Existing System	Proposed Application	Improvement
Notification	Remembered and sent manually	Alert scripts and a weekly schedule	Alerts are automatic (sent to the administrator only)
Access to data	Everyone with the spreadsheet sees everything	Roles and permission sets	Each role sees only its own components

8.4 Limitations of the Low-Code Approach

Several limits of the platform were met. The user interface can be changed only within the options of the builder, so the pages use the standard layout. The trial edition used here expires after a limited period and the separate Stage and Production environments were not configured, which meant that testing and demonstration took place on the same application. The Keyword Extraction field is not controllable: it returned no keywords for short statements and it cut some phrases, so it could not be used for rules. Alert e-mails can be sent only from and to the administrator address in the way they were built, so the notifications could not be addressed to judges and participants without inviting real users. The builder also showed small defects (Table 6.2) that had to be re-checked after saving. Finally, the application depends on one vendor: the forms, scripts and workflows live inside Zoho Creator, and moving them elsewhere would mean rebuilding them. These limits were worked around by keeping the rules in short Deluge scripts, by checking every saved setting, and by documenting the missing features instead of simulating them.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

On the no-code to pro-code spectrum, most of the application is no-code or configuration: the four forms and their field types, the lookups, the formula field, the five reports, the dashboard and its charts, the role hierarchy, the permission sets and the AI field. Low-code was needed for the seven Deluge scripts, which carry the rules that the form properties alone could not express: the duplicate checks, the range check and the e-mail messages with data from other records. No pro-code was used. A move to pro-code would be justified for a public live scoreboard, for plagiarism checking of project submissions, for integration with a college ERP, or for a very large event where

response time and API limits matter; until then, the low-code version is easier for the organisers to maintain.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project produced a working Zoho Creator application for managing a college hackathon from registration to ranking. Four connected forms hold participants, judges, teams and scores; a formula field computes totals; seven Deluge scripts and an approval process enforce the rules and send alerts to the administrator; five reports and one dashboard show the data; two permission sets limit each role to its own components; and a Keyword Extraction field tags the problem statements.

Testing with twelve participants, ten judges, twelve teams and fourteen scores confirmed that invalid scores, duplicate scores and duplicate teams are rejected and that totals, the leaderboard and the charts agree with the records. The project also showed the practical limits of low-code: no separate environments in the trial account, a weakly controllable AI field and notification limited to the administrator. The objectives of Section 1.3 were met, with the gaps stated in Table 8.1.

9.2 Future Enhancements

- Participant and judge notifications: after real users are invited, send the registration confirmation, the approval result and the scoring reminder to the participants and judges themselves, using their e-mail fields.
- Judge assignment: use the AI keywords of the problem statements and the Domain Expertise of the judges to propose which judges should score which team, and use Zia analytics for predictive insight on scores.
- Team members: add a subform to the Team form to list all members of a team, with a unique-member check, so that a participant cannot join two teams.
- Pro-code extension: use a Zoho Catalyst function and a widget for a live public scoreboard and for plagiarism checking of submitted projects.
- Deployment and portal: move to a paid edition with Stage and Production environments, and add a portal with record-level access so that participants see only their own team.

9.3 Learning Outcomes and Career Relevance

The project gave practice in the whole path of a low-code solution: modelling data as related forms, expressing rules in Deluge, automating with workflows and schedules, controlling access with roles and permission sets, using an AI field, and then testing, documenting and judging the result honestly. The skills correspond to the work of a citizen developer or low-code developer, who builds and maintains departmental applications, and to that of a solutions consultant, who must decide where low-code is enough and where pro-code is needed. The experience of reading Deluge error messages and of verifying every AI-assisted script against the running application is also relevant to the automation engineer, because it shows that generated code must always be tested before it is trusted.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] J. R. Rymer and C. Richardson, "New development platforms emerge for customer-facing applications," *Forrester Research*, Cambridge, MA, USA, Jun. 2014.

[3] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2025.

[4] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 29 October 2026).

[5] R. Sanchis, Ó. García-Perales, F. Fraile and R. Poler, "Low-code as enabler of digital transformation in manufacturing industry," *Applied Sciences*, vol. 10, no. 1, art. 12, 2020.

[6] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in *Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 171–178.

[7] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?," *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022.

[8] Devpost, "Devpost: the home for hackathons." [Online]. Available: https://devpost.com/ (Accessed: 29 October 2026).

[9] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 29 October 2026).

[10] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 29 October 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Application 'Hackathon Team Formation & Judging System' with menu sections Participants, Approvals, Teams, Judges, Scoring and Dashboard	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Name, Single Line, Drop Down, Email, Multi Line, Lookup, Number, Formula and AI Keyword Extraction fields (9 types) in the four forms	5.2	Fig. 5.2, 5.3; Tables 4.2–4.5
II	Lookup field (relationship between forms)	Y	Team.Team_Leader, Scoring.Team and Scoring.Judge	4.2, 5.2	Fig. 4.2; Table 4.1
II	Subform (parent–child data)	N	Not used; the team–judge relationship is modelled by the Scoring form with two lookups	4.2, 5.2	—
II	Field validations / layout rules	Y	Mandatory Email; range and duplicate checks in DS-01, DS-02, DS-04 (layout rules not used)	5.2	Code 5.1
II	List report with filter, grouping, custom action	Partial	Five list reports with sorting (Leaderboard); filter, grouping and custom actions not configured	5.3	Fig. 5.4; Table 5.1
II	Kanban / Calendar / Timeline / Map report	N	Not used; no date, stage or location field to support them	5.3	—
II	Pivot table / chart report	Y	Two chart components (Teams by Status, Total Score by Team) on the dashboard page	5.3	Fig. 5.5
II	Page / dashboard built with UI Builder	Y	Hackathon Dashboard page	5.4	Fig. 5.5

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge – form events (validate / success / field actions)	Y	On Validate (DS-01, DS-02, DS-04) and On Success (DS-03, DS-05, DS-06)	5.5	Code 5.1, 5.2, 5.3
II	Deluge – functions, fetch, loops, collections	Partial	Criteria fetch, fetch by ID, ifnull and string handling used; stored functions, loops and collections not used	5.5	Code 5.1, 5.2
II	Email / in-app notifications	Y	sendmail in DS-03, DS-05, DS-06, DS-07 (in-app notifications not used)	5.5, 5.6	Code 5.2, 5.3; Fig. 5.7
II	Form workflow	Y	Six form workflows (DS-01 to DS-06)	5.6	Table 5.3; Fig. 5.6
II	Scheduled workflow	Y	Judge Scoring Reminder (custom schedule, DS-07)	5.6	Table 5.3
II	Approval process	Y	Team Registration Approval (Approvals section)	5.6	Table 5.3
II	Blueprint	N	Not used; only two statuses	5.6	—
II	Batch / report workflow	N	Not used; no bulk operation required	5.6	—
II	Testing & debugging evidence	Y	Test cases, four negative tests and one debugged defect	6	Table 6.1, 6.2; Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Mail (sendmail) and Zoho Zia Keyword Extraction	5.7	Table 5.4; Fig. 5.7
III	Connection / REST API call (invokeurl)	N	Not used; no external API called	5.7	—
III	Webhook or Creator REST API	N	Not implemented	5.7	—
III	Users, roles and permission profiles	Y	Roles Judge and Participant under the CEO role; permission sets Judge Access and Participant Access (no users invited)	5.8	Fig. 5.8; Table 4.6
III	Record-level access criteria / customer portal	N	Not used; no portal and no record-level criteria	5.8	—
III	Zia / AI feature	Y	Keyword Extraction AI field Problem Keywords	5.9	Fig. 5.9
III	Analytics (charts / Zoho Analytics)	Y	Two dashboard charts; Zoho Analytics not integrated	5.9	Fig. 5.5

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Performance optimisation applied & measured	Partial	Criteria fetch used; no before/after measurement taken	7.2	Table 7.2
IV	Deployment across environments	N	Separate Stage and Production environments not configured	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Permission sets, administrator-only e-mail, validation, no stored credentials	7.3	Table 7.3
IV	Governance (audit trail, administration)	Partial	Purpose-based workflow names and workflow list as change history; audit trail and export not used	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	—
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum analysis	8.4, 8.5	Sections 8.4, 8.5
—	Mobile app / portal access	N	Not configured	5.11	—

Total features implemented: 18 / 32 (a further 4 are partly implemented: rows 6, 11, 26 and 29 of the table in order)

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	2 (Medium)	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3 (High)	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2 (Medium)	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	1 (Low)	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2 (Medium)	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Scoring Form

```
// Script ID : DS-01
// Location  : Scoring → On Validate (workflow: Validate Score Range)
// Purpose   : Reject any criterion score outside 0 to 10
if(input.Innovation > 10 || input.Technical > 10 || input.Presentation
> 10 || input.Impact > 10 || input.Innovation < 0 || input.Technical <
0 || input.Presentation < 0 || input.Impact < 0)
{
    alert "Each score must be between 0 and 10";
    cancel submit;
}

// Script ID : DS-02
// Location  : Scoring → On Validate (workflow: Prevent Duplicate
Scoring)
// Purpose   : Stop a judge from scoring the same team twice
if(input.Team != null && input.Judge != null && Scoring[Team ==
input.Team && Judge == input.Judge].count() > 0)
{
    alert "This judge has already scored this team.";
    cancel submit;
}

// Script ID : DS-03
// Location  : Scoring → On Success (workflow: Score Submitted Alert)
// Purpose   : E-mail the administrator when a score is saved
t = Team[ID == input.Team];
j = Judge[ID == input.Judge];
total = ifnull(input.Innovation,0) + ifnull(input.Technical,0) +
ifnull(input.Presentation,0) + ifnull(input.Impact,0);
sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"New score submitted for " + t.Team_Name
    message :"A score was submitted for team " + t.Team_Name + " by
judge " + j.Email + ". Total score: " + total + " out of 40."
]
info "Score alert sent for " + t.Team_Name;
```

C.2 Team Form

```
// Script ID : DS-04
// Location  : Team → On Validate (workflow: Prevent Duplicate Team)
// Purpose   : Block a second team with the same name
if(input.Team_Name != null && Team[Team_Name ==
input.Team_Name].count() > 0)
{
    alert "A team with this name already exists. Please choose a
different team name.";
    cancel submit;
}


// Script ID : DS-05
// Location  : Team → On Edit, successful submission (workflow: Notify
Team Approved)
// Purpose   : E-mail the administrator when a team is set to Approved
if(input.Status == "Approved")
{
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"Team Approved: " + input.Team_Name
        message :"Team " + input.Team_Name + " has been approved for
the hackathon."
    ]
}
```

C.3 Participant Form

```
// Script ID : DS-06
// Location  : Participant → On Success (workflow: New Participant
Alert)
// Purpose   : E-mail the administrator when a participant registers
fullName = input.Name.first_name + " " + input.Name.last_name;
sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"New participant registered: " + fullName
    message :"A new participant has registered. Name: " + fullName +
", College: " + input.College + ", Experience: " +
input.Experience_Level + ", Email: " + input.Email + "."
]
info "Participant alert sent for " + fullName;
```

C.4 Schedule

```
// Script ID : DS-07
// Location  : Schedule → Judge Scoring Reminder (custom schedule)
// Purpose   : Remind judges to finish scoring (mail goes to the
administrator)
sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"Reminder: Submit pending hackathon scores"
    message :"Judges, please complete scoring for all assigned teams."
]
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open https://creatorapp.zoho.in/vishalabs12/hackathon-team-formation-judging-system/ in a web browser and sign in with the Zoho account that the administrator has invited. The application opens with the menu on the left: Participants, Approvals, Teams, Judges, Scoring and Hackathon Dashboard. The mobile application was not configured for this project.

D.2 Instructions for the Administrator

- Step 1. Open Participants and All Participants to see the registered participants; use the Participant form to add one. The participant's e-mail is mandatory and you will receive an alert e-mail.
- Step 2. Open Judges, choose the Judge form and enter the name, domain expertise and e-mail of each judge.
- Step 3. Open Teams and choose the Team form. Enter a unique team name, the problem statement, the status and the team leader. A repeated team name is refused.
- Step 4. Open Teams and All Teams to review the teams and the keywords extracted from the problem statements. Set the status to Approved when the team is accepted, or use the Approvals section; you receive an alert e-mail for an approval.
- Step 5. Open Scoring and Leaderboard to see the ranked scores, and Hackathon Dashboard to see the two charts.
- Step 6. You will also receive an e-mail alert for each new score and a weekly reminder e-mail for the judges.

D.3 Instructions for the Judge

- Step 1. Open Scoring and choose the Scoring form.
- Step 2. Select the team in the Team field and your own name in the Judge field.
- Step 3. Enter Innovation, Technical, Presentation and Impact marks, each from 0 to 10; the Total Score is calculated for you. Click Submit. A mark outside the range, or a second score by you for the same team, is refused with a message.

- Step 4. Open Scoring Report or Leaderboard to see the scores, All Teams to see the teams and Hackathon Dashboard for the charts.

D.4 Instructions for the Participant

- Step 1. Open Participants and choose the Participant form.
- Step 2. Enter your name, college, experience level, main skill and e-mail address, then click Submit. The e-mail address is required.
- Step 3. Open All Teams to see the teams and their status, and Hackathon Dashboard to follow the standing of the teams.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	VISHAL S	Register No.	240701600
Project Title	HACKATHON TEAM FORMATION & JUDGING SYSTEM	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	VISHAL S	Register No.	240701600
Department / Section	Computer Science and Engineering	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____